



UNIT – III

Java Servlets

UNIT 3: Jave Servlets

12

Java EE Architecture - Application Servers and Containers, Servlet API and Lifecycle, Handling Client Requests and Responses - Working with Databases through Servlets-Servlet Advanced Concepts: Request Dispatcher –Session Management, Cookies and Http Session, Working with Files, Non- blocking I/O in Servlets.

Implementation

1. Create a servlet that handles HTTP file upload requests, allowing users to upload an image. Process the uploaded image server-side by applying filters (e.g., grayscale, invert colors) or transformations (e.g., resize, rotate). Use the ServletRequest and ServletResponse objects to manage the upload and return the processed image as a response. Include session tracking to remember user-specific settings (e.g., preferred filters) across multiple requests. Configure the servlet using the web.xml file, defining mappings and servlet initialization parameters.
2. Create a Java Servlet application that allows users to submit feedback through a web form and displays the submitted information on a response page.

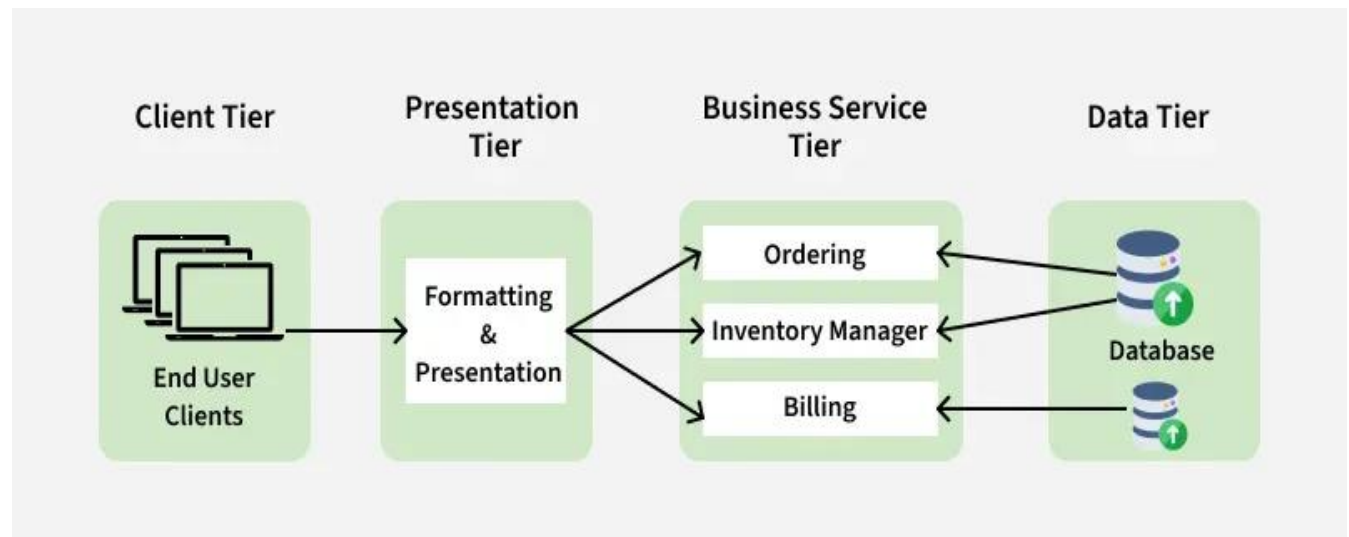
Java EE Architecture

Java Enterprise Edition (Java EE) also known as Jakarta EE which used for designing large java buildings and powerful application used by organizations. It provides tools and features that help developers to create websites, applications and backend system that can handle many users at the same time.

JEE include built in support for web development, database management and security. JEE has become the backbone of many banking, e-commerce, and organization applications due to its reliability, flexibility, and platform independence.

Architecture of Java Enterprise Edition(JEE)

Java EE follows multi-tier architecture which means the application is divided into different layers. This makes it easier to manage, scale and update without effecting the whole system.



1. Client layer

- This is a frontend layer where users interact with the application.
- It can be a web browser, mobile app or desktop application.
- It uses technologies like [HTML](#), [CSS](#), [JavaScript](#) and **JSP** (Java Server Pages).

2. Web layer (Servlet layer)

- **Web layer** handles the users request and responses.
- Uses **Servlets**, **JSP**, **JSF** to process the data from users.
- Sends data to the **business layer for processing**.

3. Business layer (Logic layer)

- **Business layer** contains the core logic of an application.
- Uses **EJB** (Enterprises JavaBeans) to handle tasks like calculations, transactions and security.



- Processes data and communicate with the database layer.

4. Data layer

- **Data layer** stores and retrieves data from a [database](#).
- Uses [JPA \(Java Persistence API\)](#) and **JDBC** for database management.
- Data layer ensures secure and efficient data storage.

5. Integration layer (Optional)

- It is used to connect the application with external systems, [API's](#) and services.
- It uses **Java Messaging Services(JMS)** and web services.

Core Components of JEE

Java EE is built on a set of core components that help developers to create scalable, secure and efficient enterprise application.

1. Servlets
2. JavaServer Pages (JSP)
3. JavaServer Faces (JSF)
4. Enterprise JavaBeans (EJB)
5. Java Message Service (JMS)
6. Java Database Connectivity (JDBC)

Application Servers

An **Application Server** is a software framework that provides an environment to **deploy, run, and manage enterprise applications**.

Features of Application Server:

- Runs **Java EE components** (Servlets, JSP, EJB, JPA, JMS, etc.)
- Provides **security, transaction management, concurrency, resource pooling**
- Supports **web services** (REST / SOAP)
- Provides **containers** for different components



Application Server	Type
GlassFish	Open Source
WildFly	Open Source
JBoss EAP	Commercial
WebLogic	Commercial
WebSphere	Commercial
TomEE	Open Source
Payara	Open Source
Geronimo	Open Source

Containers

A **Container** is a *runtime environment* provided by an Application Server. It **manages Java EE components**, including their lifecycle, security, multithreading, and resources.

List of Java EE Containers

1. Web Container (Servlet Container)
2. EJB Container (Enterprise Bean Container)
3. Application Client Container (ACC)
4. Applet Container

Web Container (Servlet Container)

A Web Container manages **web components** such as **Servlets, JSP, Filters, and Listeners**. It handles **HTTP requests and responses**, manages the **lifecycle** of web components, and provides session management.

EJB Container (Enterprise Bean Container)

An EJB Container manages **Enterprise Java Beans (EJB)** which contain business logic. It provides services like **transactions, security, concurrency, remote access, and lifecycle management** for EJBs.

Application Client Container (ACC)



An ACC runs **Java standalone client applications** that access services on the server such as EJB, JMS, and JNDI.

It provides necessary libraries and manages communication with the application server.

Applet Container

An Applet Container runs **Java Applets** inside a web browser or applet viewer.

It controls the **lifecycle** of applets (init(), start(), stop(), destroy()) and provides a secure environment.

Examples of Each Container

Container	Examples
Web Container	Apache Tomcat, Jetty, Undertow
EJB Container	WildFly, GlassFish, WebLogic, WebSphere
Application Client Container (ACC)	GlassFish ACC
Applet Container	Browser JVM, AppletViewer

Servlet

A Servlet is a Java program that runs on a server to handle client requests and generate dynamic responses. The Servlet life cycle is entirely managed by the Servlet container, which uses the `jakarta.servlet.Servlet` interface to manage Servlet objects. Understanding the life cycle helps in writing efficient and resource-safe web applications.

Note: In Jakarta EE 9 and above, use `import jakarta.servlet.*` instead of `javax.servlet.*`.

States of the Servlet Life Cycle

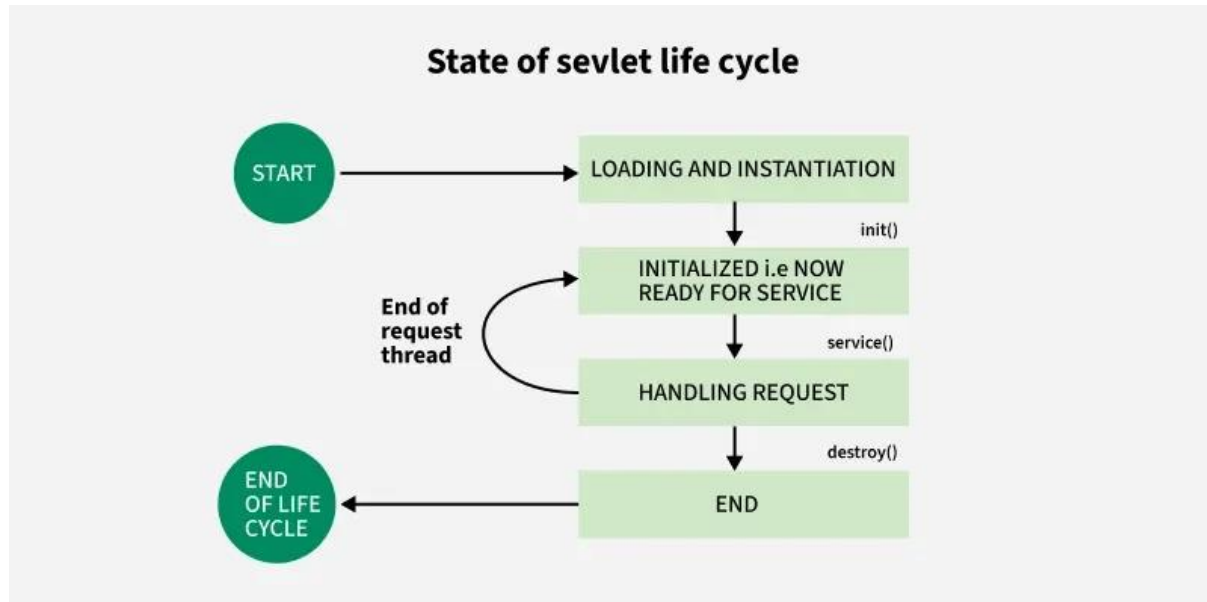
The Servlet life cycle consists of four main stages:

- Loading a Servlet
- Initializing a Servlet
- Handling Client Requests
- Destroying a Servlet

1. The servlet is initialized by calling the **init()** method.



2. The servlet calls **service()** method to process a client's request.
3. The servlet is terminated by calling the **destroy()** method.
4. Finally, servlet is garbage collected by the garbage collector of the JVM.



init() — Initialization

- Called **only once** when servlet is first loaded.
- Used for setup work:
 - Database connection
 - Reading configuration

Example:

```
public void init() {
    // initialization code
}
```

service() — Request Handling

- Called for **each request**.
- Dispatches requests to `doGet()` or `doPost()` in `HttpServlet`.

Example:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) {
```



```
// handle GET request  
}
```

destroy() — Cleanup

- Called **only once** before servlet is removed from memory.
- Used for cleanup:
 - Closing DB connections
 - Releasing resources

Example:

```
public void destroy() {  
    // cleanup code  
}
```

Servlet API

The Servlet API provides the interfaces and classes required to create servlets in Java.

It is located in:

javax.servlet

javax.servlet.http

The Servlet API provides classes and interfaces such as **Servlet**, **GenericServlet**, **HttpServlet**, **ServletRequest**, **ServletResponse**, **ServletConfig**, and **ServletContext** for building web applications.

Servlet Interface

- The main interface for all servlets.
- Defines lifecycle methods:
 - `init()`
 - `service()`



- `destroy()`

GenericServlet (Abstract Class)

- Implements **Servlet** interface.
- Protocol-independent (not HTTP-specific).
- We extend this for non-HTTP servlets.

HttpServlet (Abstract Class)

- Extends **GenericServlet**.
- Used for **HTTP-based** requests.
- Provides methods:
 - `doGet()` - visible
 - `doPost()` – encrypted format
 - `doPut()`
 - `doDelete()`

ServletRequest

- Contains request information.
- Example: parameters, headers, input stream.

ServletResponse

- Sends response back to the client.
- Example: `PrintWriter`, output stream.

ServletConfig

- Provides servlet configuration (init parameters).

ServletContext

- Gives information about the whole web application.

Handling Client Requests and Responses

When a user sends a request from a browser (client), the Web Container (e.g., Tomcat) processes it and sends it to the servlet.

The servlet then reads the request, performs some logic, and sends a response back to the browser.



This process uses two important objects:

- HttpServletRequest → For handling client requests
- HttpServletResponse → For sending responses

1. Handling Client Requests (HttpServletRequest)

The HttpServletRequest object contains all the information sent by the client, such as:

Request data you can read:

1. URL / URI
2. Form parameters
3. Query string values
4. Header information
5. Cookies
6. Client IP address

Methods of HttpServletRequest

Method	Purpose
getParameter(String name)	Gets form values (text fields, buttons)
getParameterValues(String)	Gets array of values (checkbox group)
getHeader(String)	Reads header information
getCookies()	Reads cookies
getMethod()	Returns GET / POST
getRequestURI()	Full URI

2. Handling Server Response (HttpServletResponse)



The HttpServletResponse object is used to send data back to the browser.

- HTML output
- JSON
- Files (PDF, images)
- Redirects
- Cookies

Common Methods of HttpServletResponse

Method	Purpose
setContentType(String)	Sets MIME type (text/html, json, etc.)
getWriter()	Sends text/HTML to browser
sendRedirect(String url)	Redirects to another page
addCookie(Cookie)	Adds cookies to client
setHeader(String, String)	Sets response header

```

TestWeb
|
├── index.html      ← Your HTML file goes here
|
├── src
|   └── TestingServlet.java
|
└── WEB-INF
    ├── classes      ← compiled .class files go here
    └── web.xml
  
```



LoginServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        res.setContentType("text/html"); //specify front end format
        PrintWriter out = res.getWriter();

        // Getting client request parameters
        String uname = req.getParameter("uname");
        String pwd = req.getParameter("pwd");
        // Sending response
        out.println("<h2>Welcome, " + uname + "</h2>");
    }
}
```

Web.xml

```
<web-app>

    <servlet>
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>
```



```
<servlet-mapping>  
    <servlet-name>LoginServlet</servlet-name>  
    <url-pattern>/test</url-pattern>  
</servlet-mapping>  
  
</web-app>
```

test.html

```
<form action="LoginServlet" method="post">  
    Username: <input type="text" name="uname"><br>  
    Password: <input type="password" name="pwd"><br>  
    <input type="submit" value="Login">  
</form>
```

← → ↻ ⓘ localhost:8080/MyApp/LoginServlet.html

Username:

Password:

Hello Sam

Working with Databases through Servlets

In Java web applications, **servlets** act as the middle layer between the client (browser) and the server-side resources such as databases. When a user submits a form or requests information, the servlet receives the request, processes it, and interacts with the **database** to store, retrieve, update, or delete



data. After performing the required database operations, the servlet sends an appropriate response back to the client, usually as an HTML page.

JDBC Driver Types

A JDBC driver can be fall into one of the 4 types: Type 1, Type 2, Type 3, and Type 4. The different types of JDBC drivers are mentioned below—

1. JDBC-ODBC Bridge driver [Type 1 Driver]
2. Native-API Partly-Java driver [Type 2 Driver]
3. Network Protocol Pure-Java driver [Type 3 Driver]
4. Native Protocol Pure-Java driver [Type 4 Driver]

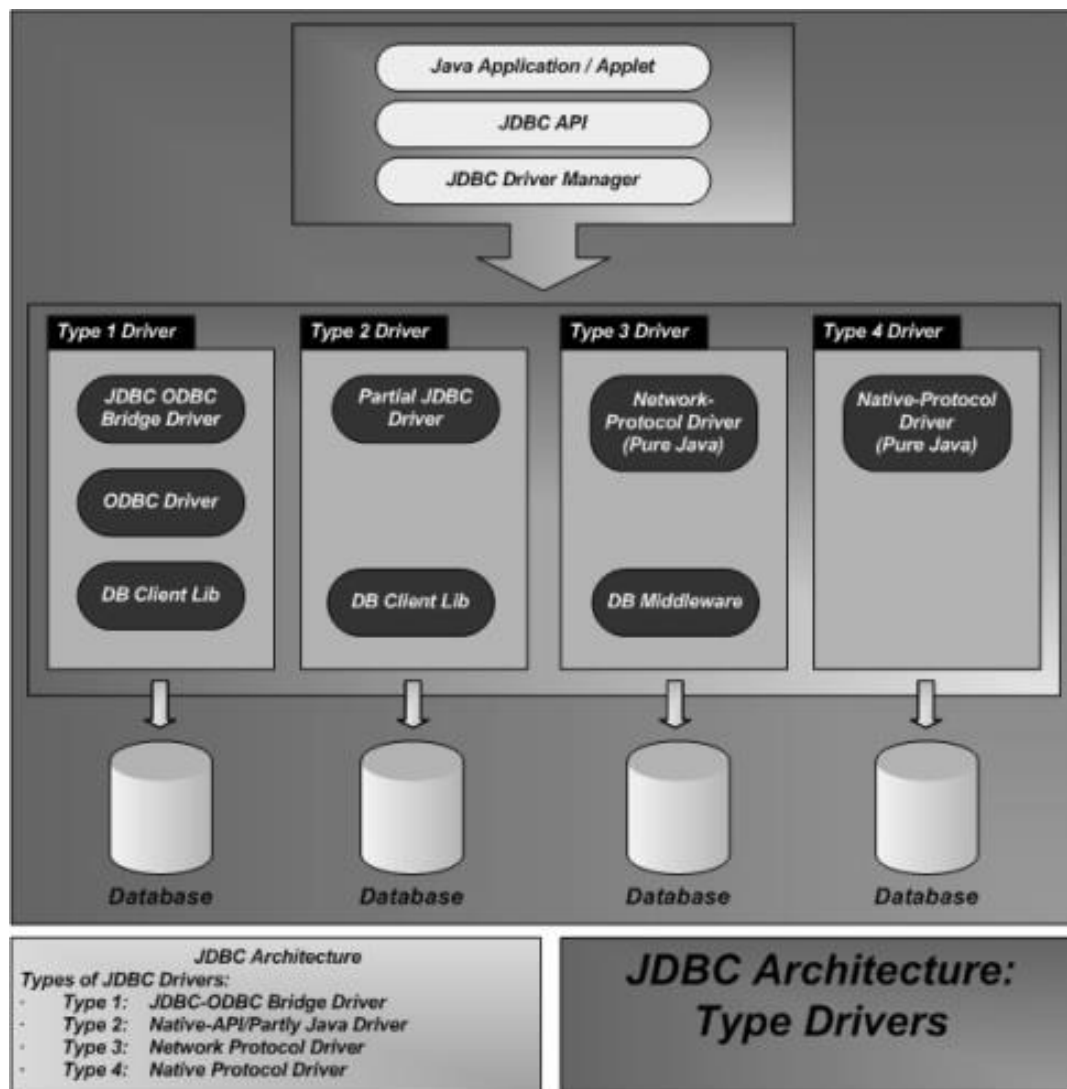
Each of the types is briefly discussed in the following segments:

1) JDBC-ODBC Bridge driver converts JDBC calls to *Open Database Connectivity (ODBC)* calls. This data access method requires that the ODBC drivers be installed on the client machines.

2) Native-API Partly-Java driver converts JDBC calls into calls in the native DBMS protocol. Since this conversion takes place on the client side, some binary code must be installed on the client machine. Type 2 drivers are drivers written in part Java and part native API (C or C++) to convert JDBC calls into calls on the client API for Oracle, MySQL, DB2, or other RDBMS.

3) Network Protocol Pure-Java driver converts JDBC calls into a net protocol that is independent of any native DBMS protocol. Then, middle-ware software running on a server converts the net protocol to the native DBMS protocol. Since this conversion takes place on the server side, no installation is required on the client machine.

4) Native Protocol Pure-Java driver converts JDBC calls into a native DBMS protocol. Since this conversion takes place on the server side, no installation is required on the client machine. Some of the Type 4 drivers for various databases are *MySQL, Oracle, MS SQL Server, and DB2*.



JDBC Connection Establishment Statement

1. Load the JDBC Driver (Optional for newer versions)

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Description:

This step loads the MySQL JDBC driver into memory so Java can communicate with the MySQL database.

Modern JDBC auto-loads the driver, but including this is safe.



2. Establish Database Connection

```
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb",  
"root",  
"password"  
);
```

- `jdbc:mysql://` → MySQL protocol
- `localhost` → MySQL server running on your system
- `3306` → default MySQL port
- `testdb` → your database name
- `"root"` → username
- `"password"` → your MySQL password

Description:

This line establishes a connection between your servlet and the MySQL database. You must specify:

- **URL** of server + database name
- **Username**
- **Password**

3. Write SQL Query with Placeholders (?)

```
String sql = "SELECT * from students";
```

4. Create a PreparedStatement Object

```
PreparedStatement pst = con.prepareStatement(sql);
```

Description:

PreparedStatement allows you to select the values from the table.

6. Execute the Query

For INSERT, UPDATE, DELETE

```
int rows = pst.executeUpdate();
```

**Description:**

`executeUpdate()` returns the **number of rows affected**.

For SELECT

```
ResultSet rs = pst.executeQuery();
```

Description:

`executeQuery()` returns a **ResultSet** which contains the fetched table data.

7. Process the Result (Only for SELECT)

```
while (rs.next()) {  
    String studentName = rs.getString("name");  
}
```

Description:

`rs.next()` moves row by row inside the result table.
You can extract columns by name or index.

8. Close All Connections

```
rs.close();  
pst.close();  
con.close();
```

Description:

Closing resources prevents memory leaks and server overload.

Example for inserting the values into the table using Servlet:

Database Table

In MySQL:

```
CREATE DATABASE testdb;  
  
USE testdb;  
  
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```




index.html

```
<!DOCTYPE html>

<html>

<head>

    <title>Database Form</title>

</head>

<body>

    <h1>Enter User Information</h1>

    <form action="_DatabaseServlet" method="post">

        Name: <input type="text" name="name"><br><br>

        Email: <input type="text" name="email"><br><br>

        <input type="submit" value="Save">

    </form>

</body>

</html>
```

DatabaseServlet.java

```
import java.io.*;
```



```
import javax.servlet.*;

import javax.servlet.http.*;

import java.sql.*;

public class DatabaseServlet extends HttpServlet {

    public void doPost(HttpServletRequest req, HttpServletResponse res)

        throws ServletException, IOException {

        res.setContentType("text/html");

        PrintWriter out = res.getWriter();

        String name = req.getParameter("name");

        String email = req.getParameter("email");

        Connection con = null;

        PreparedStatement ps = null;

        try {

            // Load Driver

            Class.forName("com.mysql.cj.jdbc.Driver");
```



```
// Connect to DB

con = DriverManager.getConnection(

    "jdbc:mysql://localhost:3306/testdb", "root", "password");

// Insert query

String query = "INSERT INTO users(name, email) VALUES (?, ?)";

ps = con.prepareStatement(query);

ps.setString(1, name);

ps.setString(2, email);

int rows = ps.executeUpdate();

if (rows > 0) {

    out.println("<h2>Data Inserted Successfully</h2>");

} else {

    out.println("<h2>Failed to Insert Data</h2>");

}

} catch (Exception e) {

    out.println("<h3>Error: " + e + "</h3>");

} finally {

    try { ps.close(); } catch (Exception ignore) {}
```



```
try { con.close(); } catch (Exception ignore) {}  
  
}  
  
}  
  
}
```

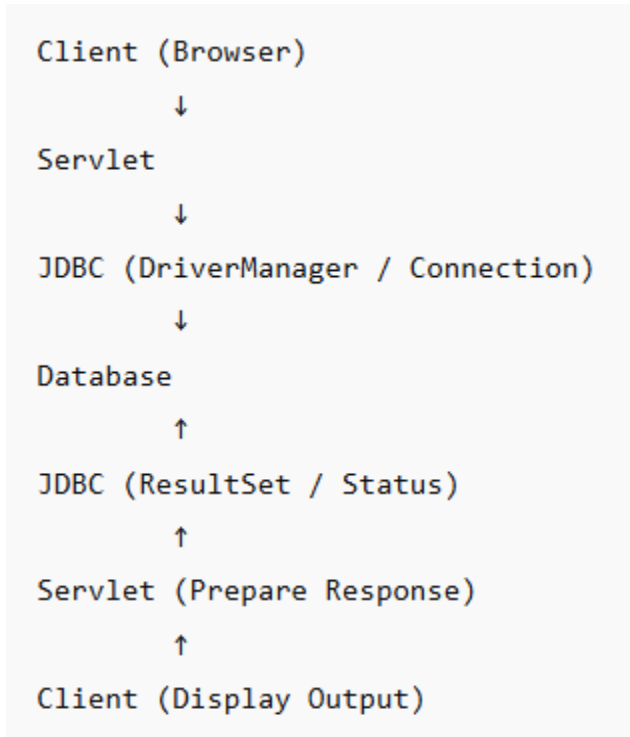
web.xml configuration

WEB-INF/**web.xml**:

```
<web-app>  
  <servlet>  
    <servlet-name>DatabaseServlet</servlet-name>  
    <servlet-class>DatabaseServlet</servlet-class>  
  </servlet>  
  
  <servlet-mapping>  
    <servlet-name>DatabaseServlet</servlet-name>  
    <url-pattern>/index</url-pattern>  
  </servlet-mapping>  
</web-app>
```

← → ↺ 🏠 🔍 localhost:8080/MyApp/test.html

Data inserted successfully



Explanation of Each Step

- 1. Client sends request**
User fills an HTML form and clicks submit.
- 2. Servlet receives request**
Servlet reads form data using `request.getParameter()`.
- 3. Servlet logic**
Validates the data before sending to the database.
- 4. JDBC Connection**
Servlet loads the JDBC driver and gets connection using `DriverManager.getConnection()`.
- 5. Execute SQL Query**
Servlet uses `PreparedStatement` to send SQL commands.
- 6. Database returns ResultSet**
Database sends back results (success/failure/data).
- 7. Servlet processes results**
For example, checks if user exists, or displays data.



8. Response sent to client

Servlet sends an HTML response (success page/error page).

Using Servlet Code for JDBC to display the content on the Screen from the Database

First, we create a directory for our web application named JdbcWeb under "C:\apache-tomcat-8.5.37\webapps". This directory consists of two sub-directories namely **src** and **WEB-INF**. A Servlet program is created with the name JdbcServlet.java to make the JDBC connection. It is placed within the sub-directory src under JdbcApp. The **WEB-INF** sub-directory consists of the Deployment Descriptor (DD) i.e. web.xml and two sub-sub-directories namely **classes** and **lib**. Basically, classes contains the .class files which are to be created during compilation along with the package structure. Here, lib has the JDBC Type 4 driver for MySQL 8.0.15 named mysql-connector-java-8.0.15.jar. The whole directory structure is shown below:

```
TestWeb
|
|— index.html
|
|— src
|   └─ DatabaseServlet.java
|
|— WEB-INF
|   └─ classes
|       └─ lib
|           └─ mysql-connector-j-8.3.0.jar
|
|— web.xml
```

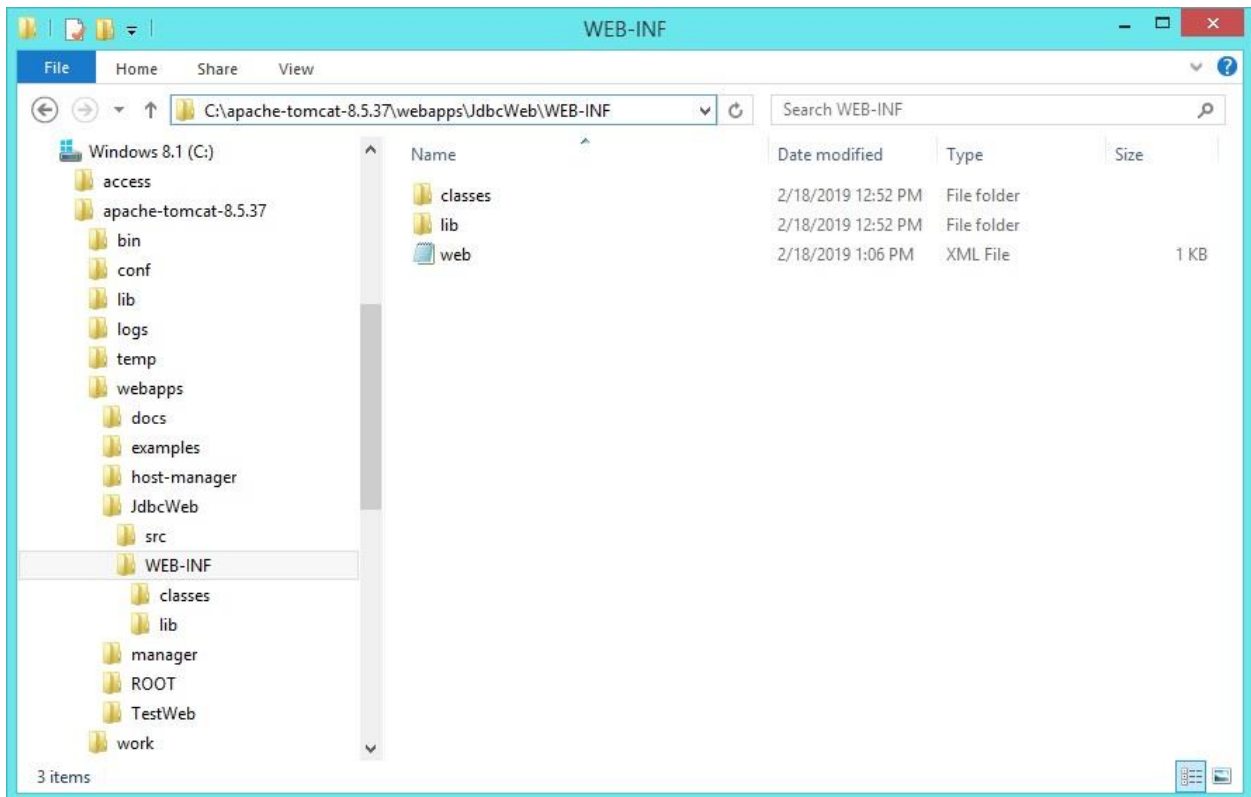


Figure 1: The directory structure of the web application

The Deployment Descriptor (DD) i.e. web.xml for our Java Servlet application is given below:

```
<web-app>
  <servlet>
    <servlet-name>JdbcServlet</servlet-name>
    <servlet-class>com.example.JdbcServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>JdbcServlet</servlet-name>
    <url-pattern>/test.do</url-pattern>
  </servlet-mapping>
</web-app>
```



The complete Servlet code is given below:~

```
import javax.servlet.*;  
import javax.servlet.http.*;  
import java.io.*;  
import java.sql.*;
```

```
//JdbcServlet.java
```

```
public class JdbcServlet extends HttpServlet {
```

```
    public void processRequest(HttpServletRequest request,  
    HttpServletResponse response)  
    throws ServletException, IOException {
```

```
        Connection con = null;  
        Statement stmt = null;  
        ResultSet rs = null;
```

```
        PrintWriter out = response.getWriter();  
        response.setContentType("text/html");  
        String url =
```

```
"jdbc:mysql://localhost:3306/test?useSSL=false&allowPublicKeyRetrieval  
=true";
```

```
        String user = "root";  
        String pass = "system";
```

```
        try {
```

```
            Class.forName("com.mysql.cj.jdbc.Driver").newInstance();  
            con = DriverManager.getConnection(url, user, pass);  
            out.println("Successfully connected to " + "MySQL server using  
TCP/IP... + "<br>");  
            stmt = con.createStatement();  
            rs = stmt.executeQuery("select * from students");
```

```
            while (rs.next()) {
```




```
        out.println("Roll = " + rs.getInt(1) + " Name = " + rs.getString(2) +
"<br>");
    }
}
catch (Exception e) {
    out.println("Exception: " + e.getMessage());
}
finally {
    try {
        if (con != null) {
            con.close();
        }
    }
    catch (SQLException e) {}
}
}

//for "HTTP GET" method
public void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    processRequest(request, response);
}

//for "HTTP POST" method
public void doPost(HttpServletRequest request,
HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
}
```



The procedure for compiling the code and running the Tomcat service (startup.bat) from command prompt is given below:~

```
C:\> set classpath=C:\apache-tomcat-8.5.37\lib\servlet-api.jar;
```

```
C:\> cd apache-tomcat-8.5.37\webapps\JdbcWeb\src
```

```
C:\apache-tomcat-8.5.37\webapps\JdbcWeb\src> javac -d ..\WEB-INF\classes JdbcServlet.java
```

```
C:\apache-tomcat-8.5.37\webapps\JdbcWeb\src> cd ../../bin
```

```
C:\apache-tomcat-8.5.37\bin> startup.bat
```

```
Using CATALINA_BASE: "C:\apache-tomcat-8.5.37"
```

```
Using CATALINA_HOME: "C:\apache-tomcat-8.5.37"
```

```
Using CATALINA_TMPDIR: "C:\apache-tomcat-8.5.37\temp"
```

```
Using JRE_HOME: "C:\Program Files\Java\jdk1.8.0_162"
```

```
Using CLASSPATH: "C:\apache-tomcat-8.5.37\bin\bootstrap.jar;C:\apache-tomcat-8.5.37\bin\tomcat-juli.jar"
```

Now we open a web browser (Google Chrome) to send the Web URL (<http://localhost:8085/JdbcWeb/test.do>) to the Tomcat Server. The output is given below:

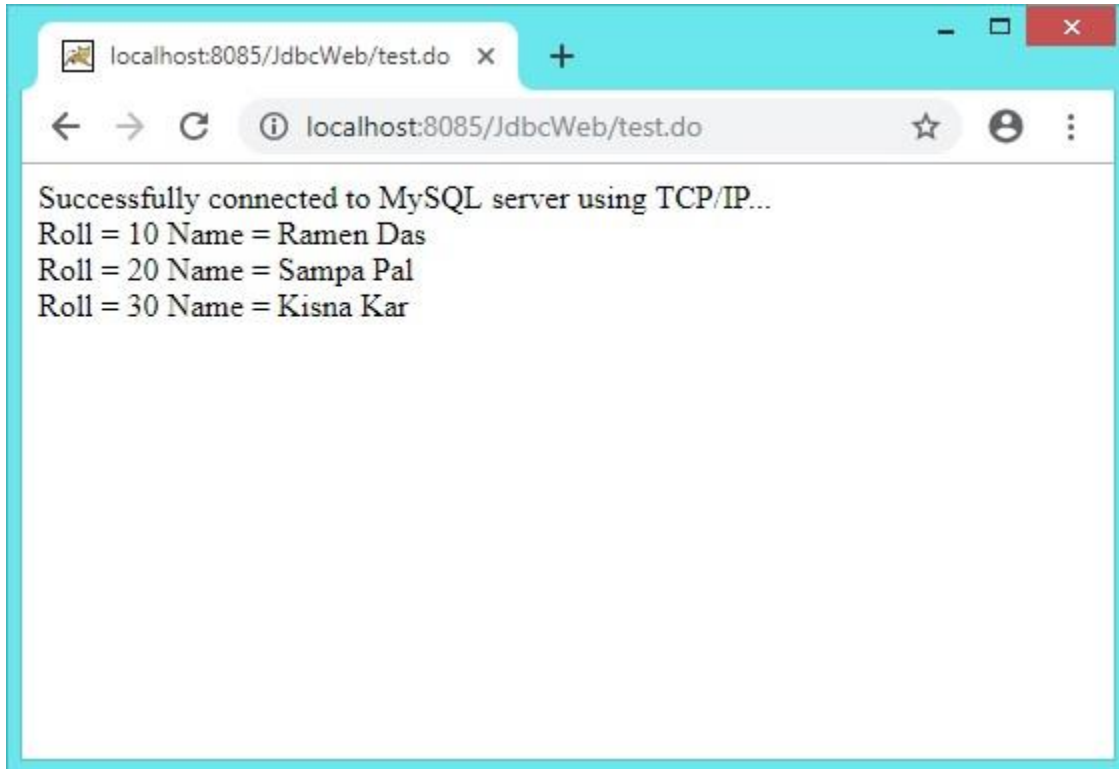


Figure 2: The output of the Java Servlet code using JDBC

Servlet Advanced Concepts: Request Dispatcher

RequestDispatcher is an interface in the javax.servlet package used to forward a request from one servlet/JSP to another resource (servlet, JSP, or HTML) within the same server.

Main Purposes

Forwarding a request to another resource

Including content of another resource in the response

How to Get RequestDispatcher

```
RequestDispatcher rd = request.getRequestDispatcher("target.jsp");
```



Methods

1. forward()

Transfers control to another resource

Same request and response objects are used

URL in browser does not change

```
rd.forward(request, response);
```

2. include()

Includes output of another resource in current response

Control returns to original servlet

```
rd.include(request, response);
```

Example

```
protected void doGet(HttpServletRequest request, HttpServletResponse  
response)
```

```
throws ServletException, IOException {
```

```
request.setAttribute("name", "Geetha");
```

```
RequestDispatcher rd = request.getRequestDispatcher("welcome.jsp");
```

```
rd.forward(request, response);
```

```
}
```



Program: (forward)

1. forward

Syntax:

`void forward(ServletRequest request, ServletResponse response) throws ServletException, IOException`

Description:

Modifier and Type:- void

This method is used to forward a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

The method get called before the response has been sent to the client. If the response is already sent then the method will throws an `IllegalStateException`.

The parameter request(`HttpServletRequest` type) and response(`HttpServletResponse` type) are the same objects as were passed to the calling servlet's service method.



This method sets the dispatcher type of the given request to DispatcherType.FORWARD.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Servforward extends HttpServlet {
    public void doPost(HttpServletRequest request,
                       HttpServletResponse response)
    {
        // Perform all the work as per your
        // application's architecture
        try {
            RequestDispatcher requestDispatcher;

            // path is a string specifying the pathname to
            // the resource. If it is relative, it must be
            // relative against the current servlet
            requestDispatcher=request.getRequestDispatcher("path");
            requestDispatcher.forward(request, response);
        }
        catch (ServletException servletException) {
```



**CHENNAI
INSTITUTE OF TECHNOLOGY**
(Autonomous)



NIRF
151 - 200 Band
Engineering 2023



```
}  
    catch (IOException ioException) {  
    }  
    catch (IllegalStateException illegalStateException) {  
    }  
}  
}
```

Program: (include)

2. include

Syntax:

`void include(ServletRequest request,ServletResponse response) throws
ServletException,IOException`

Description:



Modifier and Type:- void

This method is used to include the response of resource(for which the request passed servlet, JSP page, HTML file) in the current servlet response.

The parameter request(HttpServletRequest type) and response(HttpServletResponse type) are the same objects as were passed to the calling servlet's service method.

This method sets the dispatcher type of the given request to DispatcherType.INCLUDE.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Servinclude extends HttpServlet {
    public void doPost(HttpServletRequest request,
                       HttpServletResponse response)
    {
        // Perform all the work as
        // per your application's architecture
        try {
            RequestDispatcher requestDispatcher;

            // path is a string specifying the pathname to
            // the resource. If it is relative, it must be
```




```
// relative against the current servlet  
requestDispatcher=request.getRequestDispatcher("path");  
requestDispatcher.include(request, response);  
}  
catch (ServletException servletException) {  
}  
catch (IOException ioException) {  
}  
}  
}
```

Session Management

Session Management is the process of tracking and storing user preferences and activities during their visit to a website or application. It helps maintain continuity until the user logs out or the session expires, avoiding the need to re-enter preferences repeatedly.

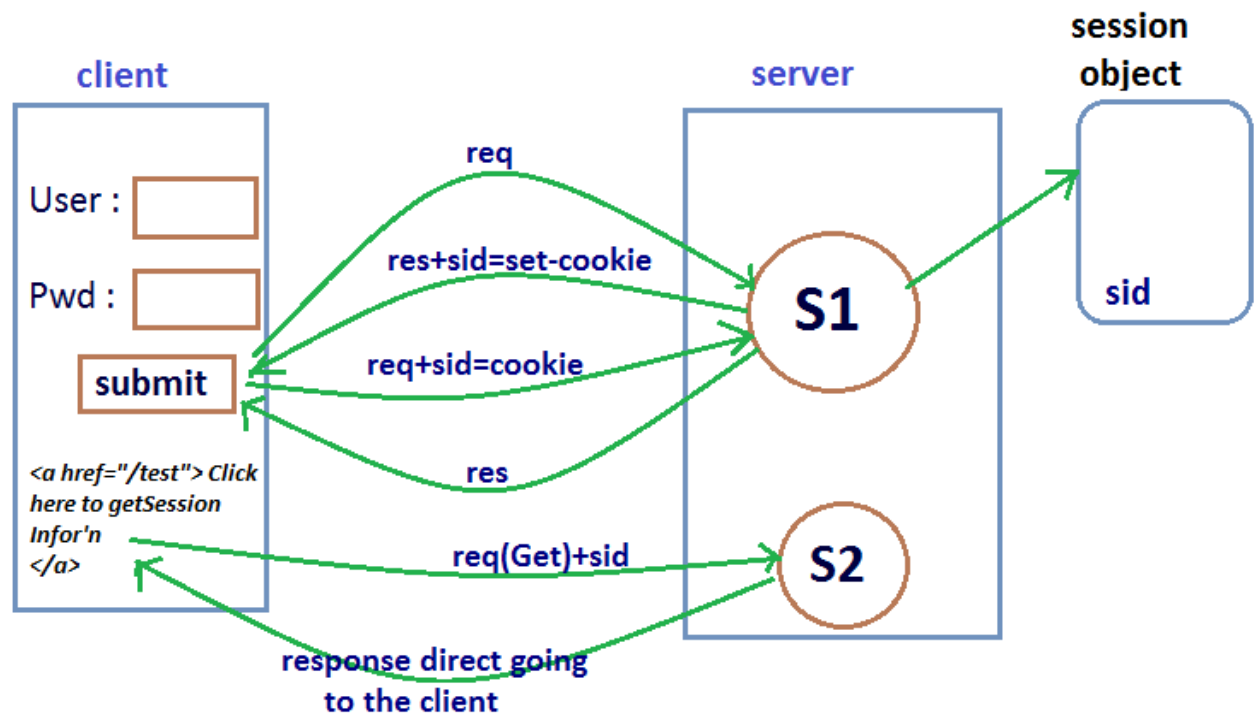
Session Management is a technique used to maintain user information across multiple HTTP requests, because HTTP is a stateless protocol.

Why Session Management is Needed

To track logged-in users

To store user-specific data (username, cart items, preferences)

To maintain continuity between requests



1. When ever client sends 1st request to the server, If the server wants to remember client information for the future purpose then it will create a Session object stores the required Session information in the form of Session scoped attributes.
2. Server sends the corresponding session-id as the part of 1st response.
3. Client saves that Session-id and send back to the Server with every consecutive request.
4. By accessing Session-id and corresponding Session object , Server can able to remember client information accross multiple requests. This mechanism is nothing but Session management by using Session-API.
5. In this mechanism entire Session information will be stored at Server side , and only Session-id will be maintained by Client.

Ex:Bank Locker



Creation of Session object :

Use `request.getSession()` to get the current session or create a new one if it doesn't exist.

```
HttpSession session = request.getSession();
```

Common Methods

1. `setAttribute(String name, Object value)`

This associates the specified value to the specified name in a particular session. This can be used to store data that needs to be managed across multiple requests.

```
HttpSession session = request.getSession();  
session.setAttribute("username", "Ram");
```

2. `getAttribute(String name)`

Returns the value which is associated with the specified name in the particular session. This is used to retrieve previously stored session attributes.

```
HttpSession session = request.getSession();  
String username = (String) session.getAttribute("username");
```

3. `removeAttribute(String name)`



It removes the attribute with the specified name from the session.

```
HttpSession session = request.getSession();  
session.removeAttribute("username");
```

4. invalidate()

Once the services are completed, it is necessary to destroy the session object. The syntax follows.

```
HttpSession session = request.getSession();  
session.invalidate();
```

Example Program:

```
import javax.servlet.http.*;
```

```
@WebServlet("/api")
```

```
public class Servsession extends HttpServlet {
```

```
    protected void doGet(HttpServletRequest request, HttpServletResponse  
response) {
```

```
        // Retrieve the HttpSession object
```

```
        HttpSession session = request.getSession();
```

```
        // Use HttpSession methods
```

```
        session.setAttribute("username", "Ram");
```

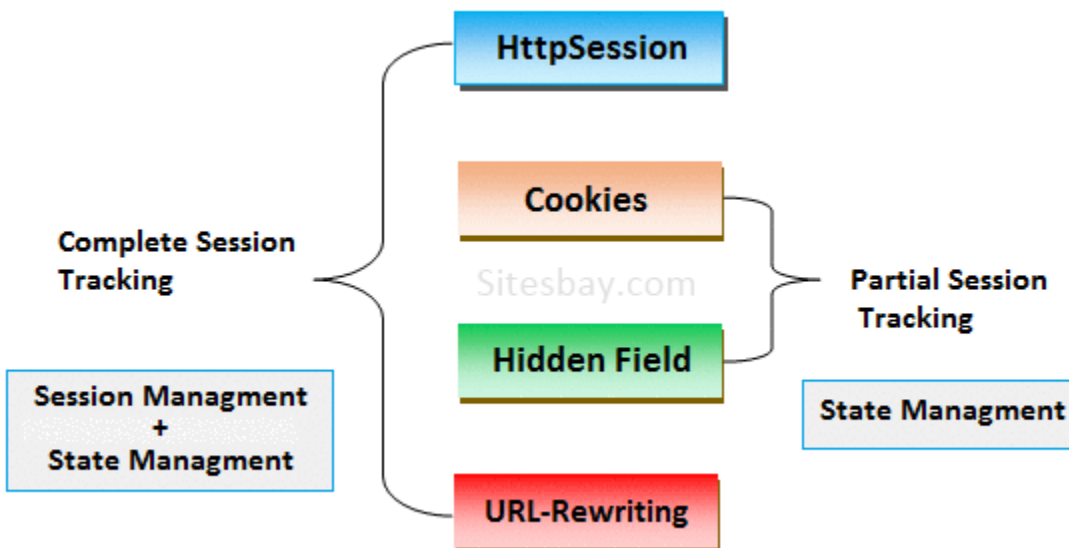
```
        String value = (String) session.getAttribute("username");
```



```
}  
  
}
```

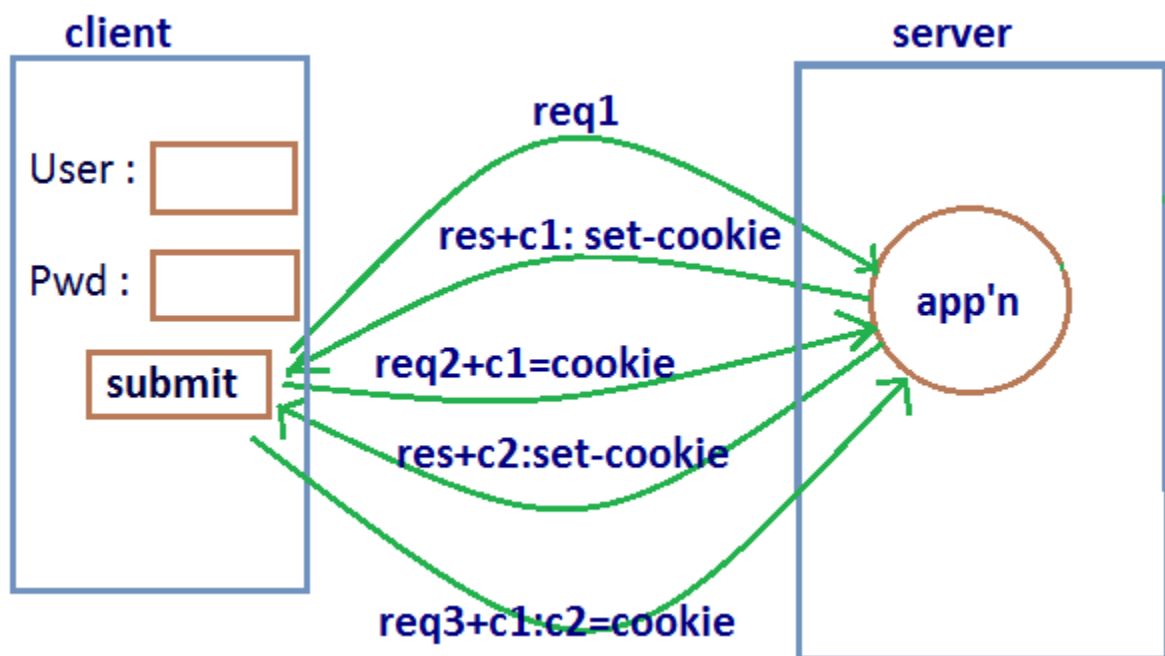
Session Management Techniques

1. Cookies
2. URL Rewriting
3. Hidden Form Fields
4. HttpSession (Most Common)



Cookies :

- ➔ Small data stored in the client's browser
- ➔ Automatically sent with every request



1. Cookie is a small amount of information (key , value) pair.
2. Whenever client sends a request to the Server , if any information required to remember for future purpose then server creates a Cookie object with that information and send that Cookie back to the client as the part of response . For, this , server uses set-cookie response header.



3. Whenever client got the response it retrieves the cookies send by the Server and stores those cookies in the local file system.
4. Client sendback all the Cookies send by the server with every consecutive request for this client use cookie request header.
5. By accessing those cookies server can able to remember client information across multiple requests.
6. We can create Cookie object by using Cookie class constructor.
Cookie cookieobject = new Cookie(String key , String value);
7. After creating the Cookie object we have to add that Cookie to the response by using addCookie().
response.addCookie(cookieobject) ;
8. At server side , we can retrieve all cookies send by the client by using get_cookies().
Cookies[] cookieobject = request.getCookies();
9. If the request doesn't associated with any Cookies then this method returns null.

Important methods of Cookie class :

public String getName()

This method returns the name of the Cookie.

public String getValue()

Returns value of the Cookie.

public int getMaxAge()

Returns the maximum age of Cookie in seconds.

public void setMaxAge(int seconds)

To set max age of cookie.

" +ve" value indicates that cookie will be expires after that how many seconds have passed.



Note that the value is max age of the cookie, when the cookie will expires , not the cookie current age.

"-ve" value indicates the cookie is not stored permanently and it will be deleted when browser exists/close.

"0" value indicates cookies to be deleted.

Demo Program for session management by cookies

login.html

```
<form action="./cookieone">
<table>
<tr><td>Name :</td><td> <input type="text"
name="uname"></td></tr>
<tr><td>Value : </td><td> <input type="text"
name="value"></td></tr>
<tr><td><input type="submit" value="submit"></td></tr>
</table>
</form>
<a href="./cookietwo">Cookie Information</a>
```

CookieServletOne.java

```
public class CookieServletOne extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
    response)
        throws ServletException, IOException {

        response.setContentType("text/html");
```




```
        PrintWriter out = response.getWriter();
        String name=request.getParameter("uname");
        String value=request.getParameter("value");

        Cookie c=new Cookie(name, value);
        c.setMaxAge(120);
        response.addCookie(c);
        out.println("Cookie added Successfully");

        out.println("domain"+c.getDomain());
        out.println("comment"+c.getComment());
        out.println("maxage"+c.getMaxAge());
        out.println("version"+c.getVersion());
        out.println("hashCode"+c.hashCode());

        RequestDispatcher
        rd=request.getRequestDispatcher("login.html");
        rd.include(request, response);
    }
}
```

URL - Rewriting :

URL Rewriting is a session management technique in which session information is appended to the URL and sent from one request to another.

1. Whenever Cookies are disabled at client side , browser unable to see "set-cookie" response header and hence browser unable to get session id and cookies send by the Server .
2. Due to this browser can't send session id & Cookies to the server and hence server is unable to remember client information across multiple requests . So that Session management fails.
3. To resolve this we should go for url rewriting technique.



4. The central idea in this technique is append required Session information to the url , instead of appending to "set-cookie" response header.
5. Whenever client clicks url for further communication server can get required Session information with the url . So that Server can able to remember client information across multiple requests.

URL Re-writing = URL + session information
OR
URL Re-writing = URL ; jsessionid=1234

Hidden Form Fields (Session Management Technique)

Hidden Form Fields are used to maintain session information by storing data in hidden HTML input elements, which are sent to the server when a form is submitted.

Syntax

```
<input type="hidden" name="userid" value="101">
```

Example

```
<form action="profile" method="post">  
  <input type="hidden" name="user" value="Geetha">  
  <input type="submit" value="Continue">  
</form>
```

Servlet Code

```
String user = request.getParameter("user");
```



How It Works

Session data is placed in a hidden field

User submits the form

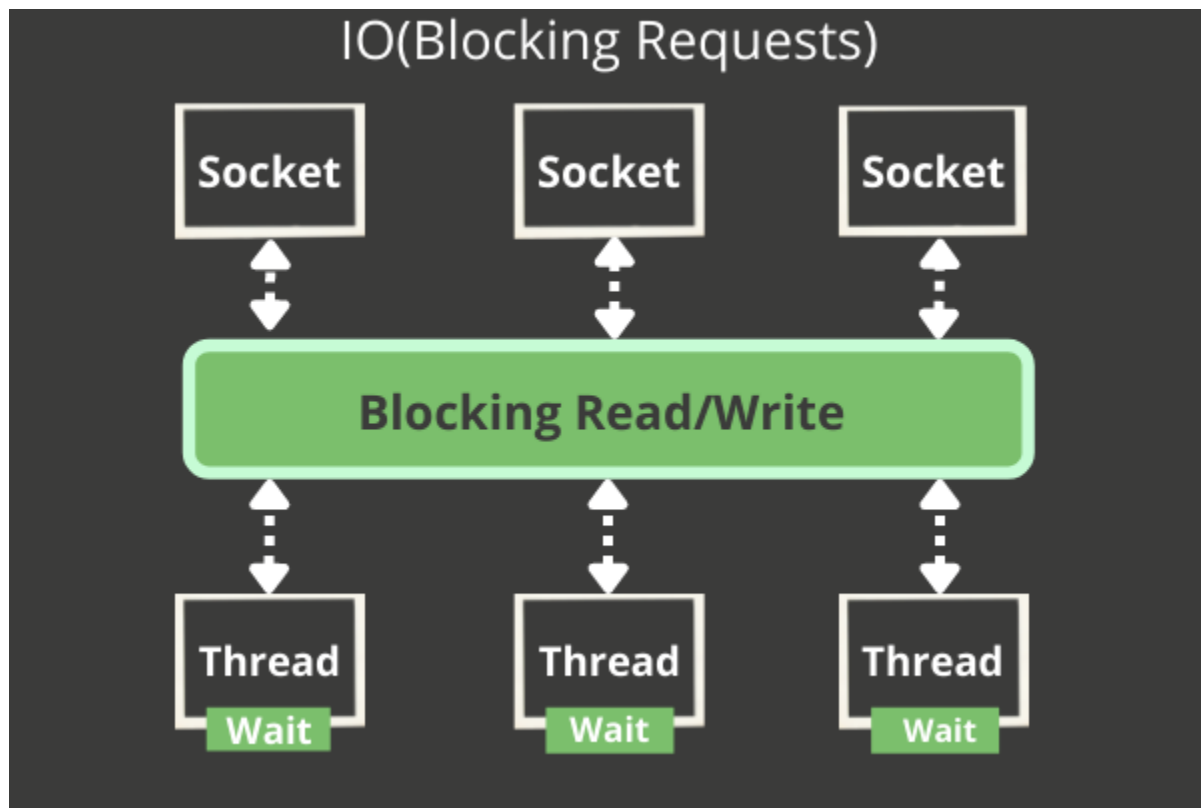
Data travels with the request

Server reads the value

A **Non-Blocking** server means that it is able to have multiple requests in progress at the same time by the same process or thread because it uses **Non-Blocking I/O**. Let's understand it with a simple example as follows: A blocking process is just like a queue at ticket counters. Each customer is one after another, no one advances until the person at the front of the queue has been served. While a non-blocking process could be thought of as a waiter at a restaurant, trying to serve the people at once, cycling through them, and handling their meals.

A. Blocking Servers

A server using a **blocking socket** works in a synchronous manner, which means whenever it gets a request it completes that and then serves for the other. Here, multi-threading comes into the picture. Suppose there are three requests in the queue, then the server will respond to the first one and then the other, and so on. It makes the clients wait for a longer time in order to achieve the desired goal. As shown in the picture, multiple threads are created in order to serve each and every request which is a very CPU-intensive task.



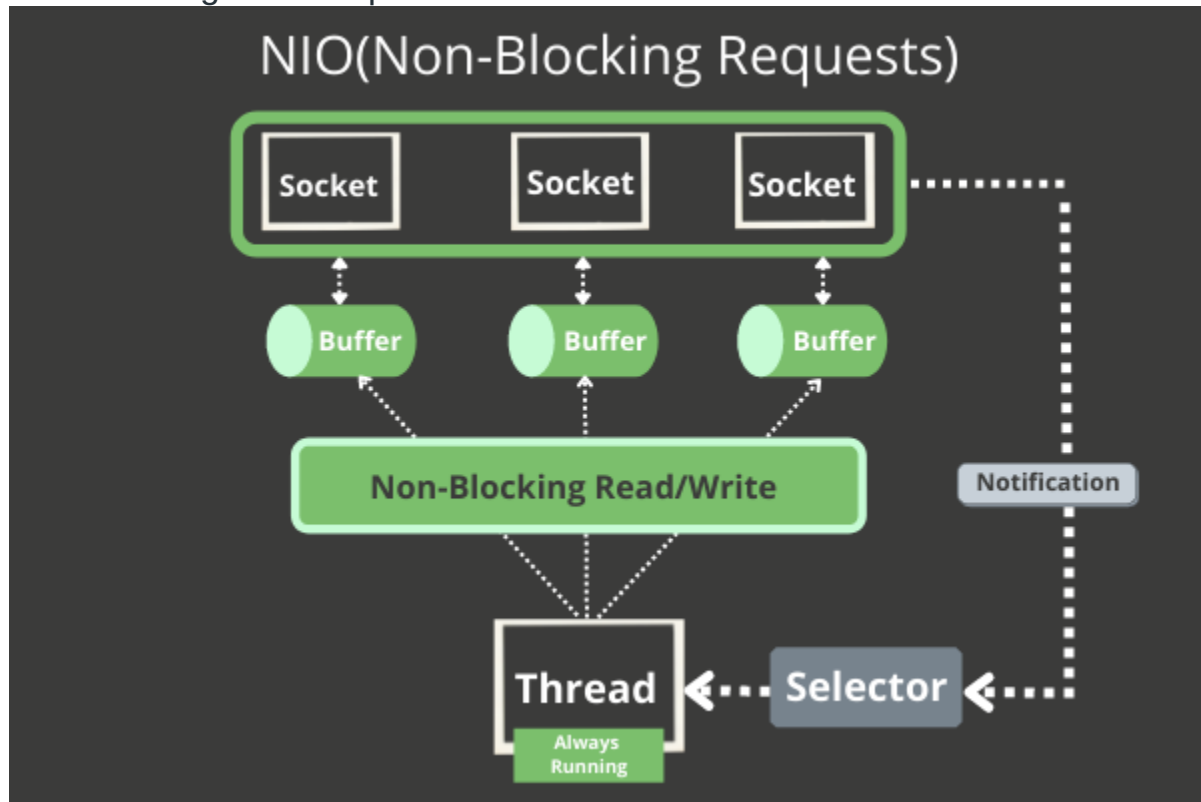
Picture depicting Server using Blocking Socket

The system calls that suspend or put the calling process on wait until the event (on which the call was blocked) occurs after which the blocked process is woken up and is ready for execution are blocking system call. For example, once a call is invoked by a thread, the mode of execution is transferred to the kernel, and the call blocks until the completion of a certain event, the particular calling thread executing in the kernel mode is blocked and is put on some waiting queue. Once the event completes, the thread wakes up and is again put on the ready queue of the CPU.

B. Non-Blocking Servers

In the **Non-Blocking** approach, one thread can handle multiple queries at a time. A server using a Non-Blocking socket works in an asynchronous manner means once the request is received it lets the system perform the task on it, and continues to take other requests and once the task is over it responds according to each request. As shown in the picture it only has one thread and the whole process is carried out using a concept called “**Event Loop**”. Here

the system is not idle and never waits for each and every task to get completed before taking other requests.



Picture depicting Server using Non-Blocking Socket

For example, take the `read()` system call that reads data from a file into the buffer allocated by the user program. The call returns into userspace only after the user buffer has been filled with the desired data. The thread is essentially put to sleep only to be later awakened when disk control has completed the I/O from the storage device and put the data into the kernel space buffer.